

ADVDIFFTS™ :

a parallel solver for 2D time dependent Advection-Diffusion equations.

Contents

1	Introduction
2	Spatial Discretization Method
3	Implementation
3.1	The PETSc library
3.2	Data Structures for the FEM method
3.3	The <i>advdiffs</i> code
4	Test Example
5	Bibliography

1 Introduction

In this document we present a parallel implementation of a Finite Element solver for 2d time dependent Advection-Diffusion problems of the form:

$$(1) \begin{cases} \partial_t u(t, x, y) - \varepsilon(\Delta u(t, x, y) - \nabla \left(\frac{\nabla \psi}{\varepsilon} u(t, x, y) \right)) = 0 & \text{in } \Omega \times [0, T] \\ u(t, x, y)|_{\partial\Omega} = 0 & \forall t \in [0, T] \\ u(0, x, y) = 0 \end{cases}$$

Note that for an Advection-Diffusion problem to be cast into this form that the advection field need be irrotational so that it can be expressed as the gradient of the scalar potential ψ .

The scheme used for the spatial discretization of the problem is the Finite Volume Scharfetter Gummel method which can be interpreted as a primal mixed finite element method where the constitutive relation for the total flux J is eliminated via an appropriate quadrature formula as extensively described in **Annex I : Conservative and Monotone Finite Element Methods for The Spatial Discretization of Diffusion-Advection-Reaction Problems.**

For the convenience of the reader, **Section 2** of the present document summarizes some of the main aspects of the FVSG method focusing on those more relevant for the implementation.

Section 3 describes in more detail the parallel implementation of the solver and the PETSc library on which it relies.

Section 4 contains the results

2 Spatial Discretization Method

To briefly describe the FVSG we applied to the spatial discretization

of (1) method we refer to the following simplified stationary problem:

$$(2) \begin{cases} -\nabla \cdot J = 0 & \text{in } \Omega \\ J = \varepsilon(\nabla u(x, y) - \frac{\nabla \psi}{\varepsilon} u(x, y)) \\ u(x, y)|_{\partial\Omega} = 0 \end{cases}$$

Given a conforming triangulation $\mathcal{T}_h$ of the computational domain Ω

The standard Galerkin/Finite Element Method applied to problem (2) reads:

find $u_h \in V_h$ such that

$$\int_{\Omega} J_h(u_h) \cdot \nabla v_h d\omega = \sum_{\tau \in \mathcal{T}_h} \int_{\tau} J_h^{(\tau)}(u_h) \cdot \nabla v_h d\omega = 0 \quad \forall v_h \in V_h$$

We chose V_h to be the set of continuous an piecewise linear functions on the triangulation $\mathcal{T}_h$.

In the FVSG method we express the total flux on each element in terms of the nodal values of the scalar unknown u_h and of the potential ψ as

$$J_h^{(\tau)}(u_h) = \frac{1}{|\tau|} \sum_{i=1 \dots 3} |e_i| s_i j_{e_i}^{SG} u_h t_i.$$

where $|e_i|$ is the length of the side opposite to node i , t_i is the tangential vector of the side e_i , $s_i = |\tau| |e_i| \nabla \phi_{i+1} \cdot \nabla \phi_{i-1}$, ϕ_i is the test function which is nonzero in the i -th node and

$$j_{e_i}^{SG} = \varepsilon u_{i+1} B(\psi_{i-1} - \psi_{i+1}) - u_{i-1} B(\psi_{i+1} - \psi_{i-1}) |e_i|$$

with the Bernoulli function defined as

$$B(z) = \frac{z}{e^z - 1}$$

The main advantage of this formulation is to yield a linear system where the coefficient matrix is an M-matrix, given that the triangulation $\mathcal{T}_h$ is Delaunay and that the convective field $\nabla \psi$ is continuous.

The local matrix for element τ in this discretization scheme can be written as:

$$A_{\tau}^{(loc)} = \begin{pmatrix} -(a_{21} + a_{31}) & a_{12} & a_{13} \\ a_{21} & -(a_{12} + a_{32}) & a_{23} \\ a_{31} & a_{32} & -(a_{23} + a_{13}) \end{pmatrix},$$

$$a_{12} = B\left(\frac{\psi_2 - \psi_1}{\varepsilon}\right) L_{12}$$

$$a_{13} = B\left(\frac{\psi_1 - \psi_3}{\varepsilon}\right) L_{13}$$

$$a_{21} = B\left(\frac{\psi_1 - \psi_2}{\varepsilon}\right) L_{12}$$

$$a_{23} = B\left(\frac{\psi_3 - \psi_2}{\varepsilon}\right) L_{23}$$

$$a_{31} = B\left(\frac{\psi_1 - \psi_3}{\varepsilon}\right)L_{31}$$

$$a_{32} = B\left(\frac{\psi_2 - \psi_3}{\varepsilon}\right)L_{32}$$

where L_{ij} are the local matrix entries for the P^1 discretization of the Laplace operator.

3 Implementation

To implement the advdiffs solver we relied on the PETSc numerical library and on the METIS mesh partitioning tool.

PETSc is a suite of data structures and routines for the scalable (parallel) solution of scientific applications modeled by partial differential equations. It employs the MPI standard for all message-passing communication.

As the structure of the solver is strongly influenced by the structure of PETSc we start this section on implementation briefly describing this latter in subsection 3.1.

PETSc was developed bearing in mind primarily the needs of PDE solvers based on finite difference schemes, therefore, although it has specific tools that help in dealing with indexing degrees of freedom of unstructured meshes in parallel, the data structures used to deal with the triangulation had to be built from scratch. A description of such data structures is in Section 3.2. Finally Section 3.3 contains appropriately commented noteworthy excerpts of the advdiffs source code.

3.1 The PETSc library

Figure (1) represents the Organization of the PETSc library.

The approach with which it is built is to create a level of abstraction in between user implemented PDE solver application and lower level tools as the time stepping (ODE solver) algorithms, linear algebra routines (vector/matrix storage/operations, iterative linear system solvers, preconditioners etc.) and the basic message passing routines for parallelization.

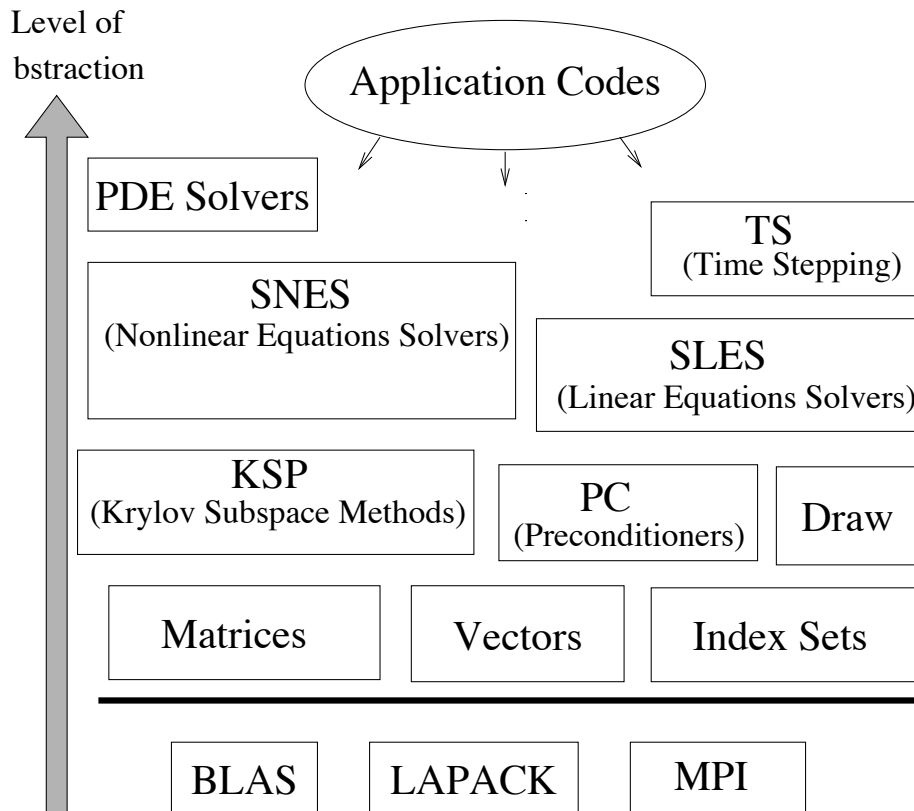


Figure 1: Organization of the PETSc Libraries

In general a user should only make use of the highest level blocks in the PETSc organization diagram. I.e. in our case, as we want to solve a time dependent problem, we should, in principle, only make use of the TS time stepping module, which in turn should automatically call the KSP linear system solving module if needed (i.e. if an implicit time stepping algorithm is selected).

Unfortunately no high-level module is provided to deal with unstructured mesh data, so we had to build our own code for this purpose using the PETSc Index Set routines. Below we describe briefly the main PETSc modules we directly or indirectly used.

TS

The Time Stepping module is an abstract interface to the ODE solver routines implemented in PETSc.

Euler, Backward-Euler and Crank-Nicholson schemes are implemented in the library while more methods are available by linking to an external ODE solver (PVODE). The particular scheme to use can be chosen at run-time while the application code is as independent as possible of the particular algorithm choice.

KSP

Iterative Linear system solvers based on Krylov subspace methods.

The particular algorithm may be chosen at run-time while the application code is independent of the solver chosen. The KSP module is called directly from the TS module when a matrix inversion is needed to advance in time.

AO

Application ordering is a low level module we used to convert from the numbering of degrees of freedom in the input mesh structure to that used by the parallel code where nodes are assigned in blocks to different processors and numbered so that each processor owns a contiguous set of indices.

IS

The index set module helps converting indices from a global numbering scheme (e.g. the application or input/output schemes referred to above) to a numbering scheme local to each processor.

3.2 Data Structures for the FEM method

In this section we describe the data structures used in the advdiffs code to deal with data defined on an unstructured triangular mesh.

A triangle in the mesh is represented by the Ttriangle structure defined as follows:

```
typedef struct {
    PetscInt  vertex[3], elnum;
    PetscReal area, shgx[3], shgy[3];
    PetscInt  bccode[3];
} Ttriangle;
```

vertex is the set of nodes of the triangle in the same global numbering used in the input files.

elnum is the index in input global numbering of the triangle.

area is the area of the triangle

shgx[i] and **shgy[i]** are the x and y components of the gradient of the i-th shape function restricted to the present triangle

bccode[i] is 1 if the i-th node is a boundary node and 0 otherwise

The mesh is stored in a structure called Tmesh defined as below:

```
typedef struct{
    Ttriangle      *triangles;
    AO             reordering;
    ISLocalToGlobalMapping mapping;
    VecScatter     scatter;
    PetscReal      *locx, *locy;
    Mat            P;
    int            *egen, toten, *egnn;
    int            *startnn, totnn, totlocnn;
} Tmesh;
```

Here

triangles is an array containing all triangles owned by a processor.

ao is the Application Ordering context needed to convert input/output global ordering to the application global ordering used internally to partition matrices and vectors.

mapping is the index set context used to convert indices of nodes in a numbering scheme local to a single processor to the application global ordering.

scatter is the vector scatter context needed to scatter(gather) data from(to) an array in local ordering to(from) an array in global ordering shared by all processors.

locx and **locy** are the coordinates of all nodes of all elements owned by one processor, note that this is more than just the nodes owned by the processor because internal boundary nodes are included.

P is a permutation matrix such that a given a vector v in application ordering, the vector $u = P' v$ is the same vector in input/output ordering. Similarly, to reorder a matrix one should use a transformation of the form $U = P' V P$

egen is an array containing the number of elements owned by each processor

toten is the total number of elements in the mesh

egnn is the total number of all nodes of all elements owned by each processor

startnn is the first index of nodes belonging to a processor in application ordering

totnn is the total number of nodes in the mesh

totlocnn is the total number of nodes belonging to a processor

The data defined on the mesh is stored in a structure of type Tdata:

```
typedef struct{
    PetscReal      *source;
    PetscReal      *potential;
    PetscReal      *density;
}Tdata;
```

source unused

potential the value of the potential ψ in each node belonging to this processor including internal boundary nodes

density the value of the unknown u in each node belonging to this processor including internal boundary nodes

3.3 The advdiffs code

In this section we comment the structure of the advdiffs code, the code is divided in 5 main sections:

- Initialization and reading command line options.
- Reading mesh data from file and distributing information to all processors by appropriate scatters.
- Build matrices used for the solution of linear systems in the time stepping process.
- Building the TS context and advancing in time.
- Freeing memory.

Each section begins with a call like:

```
PreLoadBegin(PETSC_TRUE, "Read Data");
```

that logs the section of the code in the final profiling report that can be activated by the command line option -log-summary <filename>

furthermore this function ensures that preloading is used (the portion) of code is executed twice to get more accurate timing. To disable preloading and gain speed the command line option -preload false should be used.

```
int main(int argc, char **argv)
{
```

< ... OMISSIS ... >

```
/*
    Read Command Line Options
*/
```

read the name of the data directory

```
PetscOptionsGetString(PETSC_NULL, "-prefix", prefix, 49, &flg);
if(!flg)
    sprintf(prefix, "nproc2/");
```

read the time step size

```
PetscReal deltat;
PetscOptionsGetReal(PETSC_NULL, "-deltat", &deltat, &flg);
if(!flg)
    deltat = (PetscReal) 0.1;
```

read the maximum number of time steps

```
PetscInt maxts;
PetscOptionsGetInt(PETSC_NULL, "-maxts", &maxts, &flg);
if(!flg)
    maxts = (PetscInt) 10;
```

read the diffusion coefficient

```
PetscReal epsilon;
PetscOptionsGetReal(PETSC_NULL, "-epsilon", &epsilon, &flg);
if(!flg)
    epsilon = (PetscReal) 0.000001;
```

```
/*
    Read mesh
*/
```

```
PreLoadBegin(PETSC_TRUE, "Read Data");
```

read node coordinates, connectivity, initial condition and potential field.

```
readmesh(&mesh, &data, &sol, prefix);
```

```
/*
    Build linear system matrices
*/
```

```
PreLoadStage("Build System");
ierr = PetscPrintf(MPI_COMM_WORLD,
```

```

        "\n\n****\nBuild linear system matrices\n****\n");
CHKERRQ(ierr);

```

Build the FEM discretization SG of the Advection-Diffusion operator and the mass matrix M

```

compsg(&mesh,&data,&SG,&M,epsilon);

VecDuplicate(sol,&Msol);
PetscObjectSetName((PetscObject) sol,"sol");
PetscObjectSetName((PetscObject) Msol,"Msol");
MatMult(M,sol,Msol);

```

```

ierr = MatScale(SG,mone*epsilon);

```

Name tags are added to objects so they are used in the MATLAB formatted output files

```

PetscObjectSetName((PetscObject) SG,"SG");
PetscObjectSetName((PetscObject) mesh.P,"P");
PetscObjectSetName((PetscObject) M,"M");

```

Disabled code to print out the matrices for debugging

```

#undef __printmat__
#ifdef __printmat__
    PetscViewerASCIIOpen(MPI_COMM_WORLD,"matrici.m",&viewer);
    PetscViewerSetFormat(viewer,PETSC_VIEWER_ASCII_MATLAB);

    MatView(SG,viewer);
    MatView(mesh.P,viewer);
    MatView(M,viewer);
    VecView(sol,viewer);
    VecView(Msol,viewer);

    PetscViewerDestroy(viewer);
#endif

/*
    Step time
*/

PreLoadStage("Step Time");

ierr = PetscPrintf(MPI_COMM_WORLD,"\n\n****\nStep time \n****\n");
CHKERRQ(ierr);

```

Build the time-stepping context:

```

ierr = TSCreate(MPI_COMM_WORLD,&ts); CHKERRQ(ierr);

```

Define the problem type i.e. linear or nonlinear

```

ierr = TSSetProblemType(ts,TS_LINEAR); CHKERRQ(ierr);

```

Define time-step, initial time and time interval duration

```

ierr = TSSetInitialTimeStep(ts,zero,deltat); CHKERRQ(ierr);
ierr = TSSetDuration(ts,maxts,(PetscReal)maxts * deltat ); CHKERRQ(ierr);

```

Set initial condition

```

ierr = TSSetSolution(ts,Msol); CHKERRQ(ierr);

```

Define the matrix used to compute time derivative at each step

```

ierr = TSSetRHSMatrix(ts,SG,SG,PETSC_NULL,PETSC_NULL); CHKERRQ(ierr);

```

Chose ODE solver algorithm, command-line options will override

```
ierr = TSSetType(ts,TS_BEULER); CHKERRQ(ierr);
```

Set up the routine to output solution at each time-step

```
ctxmonitor.mesh = &mesh;
sprintf(ctxmonitor.pref, "%s", prefix);
KSPCreate(MPI_COMM_WORLD,&(ctxmonitor.ksp));
KSPSetOperators(ctxmonitor.ksp,M,M,SAME_NONZERO_PATTERN);
KSPSetFromOptions(ctxmonitor.ksp);

ierr = TSSetMonitor(ts,monitor,(void *)(&ctxmonitor),PETSC_NULL);
CHKERRQ(ierr);
```

Override default options with those (eventually) given on command line

```
ierr = TSSetFromOptions(ts); CHKERRQ(ierr);
```

Advance in time until the max number of time steps has been performed or the time interval is over

```
PetscInt steps; PetscReal ftime;
ierr = TSStep(ts,&steps,&ftime); CHKERRQ(ierr);
```

```
/*
Clean up
*/
```

```
PreLoadStage("Clean up");
```

```
ierr = PetscPrintf(MPI_COMM_WORLD, "\n\n****\nClean up\n****\n");CHKERRQ(ierr);
```

Free memory

```
ierr=destroymesh(&mesh);CHKERRQ(ierr);
ierr=destroydata(&data);CHKERRQ(ierr);

ierr = MatDestroy(SG);CHKERRQ(ierr);
ierr = MatDestroy(M);CHKERRQ(ierr);
ierr = VecDestroy(sol);CHKERRQ(ierr);
ierr = VecDestroy(Msol);CHKERRQ(ierr);
```

```
PreLoadEnd();
```

```
ierr = PetscFinalize();CHKERRQ(ierr);
return 0;
```

```
}
```

4 Test Example

A test example is included with the distribution, it can be ran on two processors by issuing the command

```
make runexample
```

The example problem is equation (1) with

$$\Omega = (0, 1) \times (0, 1)$$

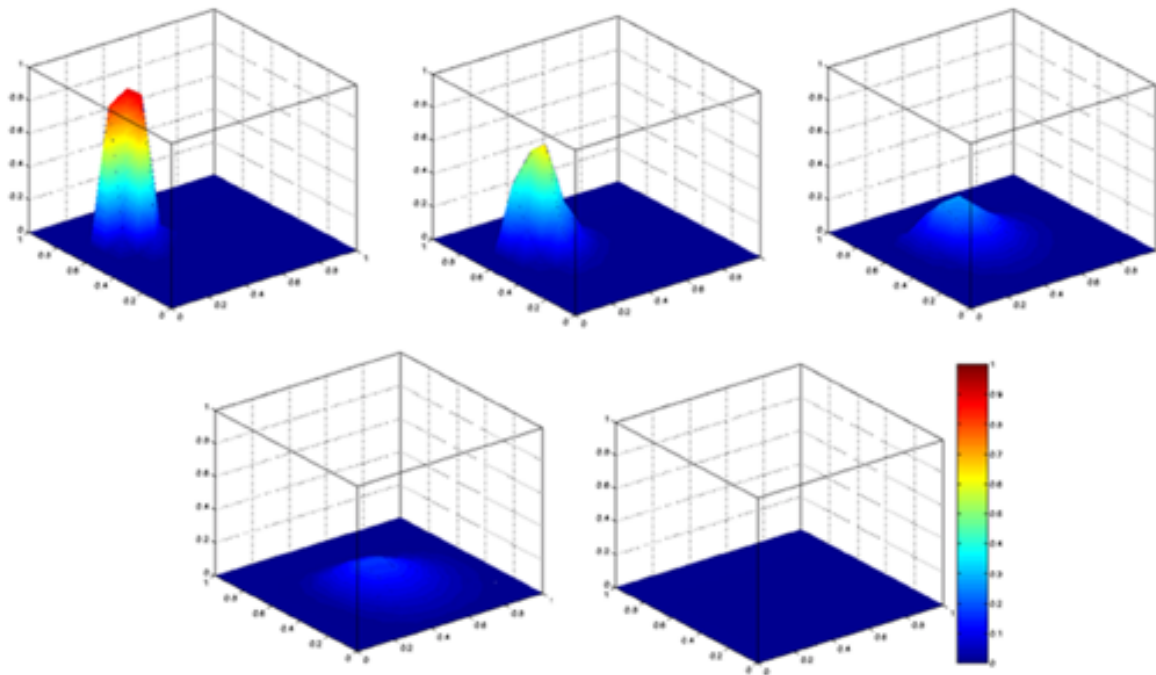
$$T = 10$$

$$\varepsilon = 10^{-6}$$

$$\psi = 10(x - y)$$

the initial condition is 1 in a circle of radius .1 in the upper-left corner of the domain and zero elsewhere.

the results are pictured in figure 2 below:



5 Bibliography

- [1] S. Balay, K. Buschelman, V. Eijkhout, W. D. Gropp, D. Kaushik, M. G. Knepley, L. C. McInnes, B. F. Smith, and H. Zhang. PETSc users manual. Technical Report ANL-95/11 - Revision 2.1.5, Argonne National Laboratory, 2004.
- [2] S. Balay, K. Buschelman, W. D. Gropp, D. Kaushik, M. G. Knepley, L. C. McInnes, B. F. Smith, and H. Zhang. PETSc Web page, 2001. <http://www.mcs.anl.gov/petsc>.
- [3] R. E. Bank, W. M. Coughran, and L. C. Cowsar. Analysis of the finite volume Scharfetter-Gummel method for steady convection diffusion equations. *Computing and Visualization in Science*, 1(3):123–136, 1998.
- [4] E. Gatti, S. Micheletti, and R. Sacco. A New Galerkin Framework for the Drift-Diffusion Equation in Semiconductors. *East West J. Numer. Math.*, 6(2):101–135, 1998.
- [5] J. Xu and L. Zikatanov. A monotone finite element scheme for convection–diffusion equations. *Math. Comp.*, 68(228):1429–1446, 1999.
- [6] S. Balay, K. Buschelman, V. Eijkhout, W. D. Gropp, D. Kaushik, M. G. Knepley, L. C. McInnes, B. F. Smith, and H. Zhang. PETSc users manual. Technical Report ANL-95/11 - Revision 2.1.5, Argonne National Laboratory, 2004.
- [7] S. Balay, K. Buschelman, W. D. Gropp, D. Kaushik, M. G. Knepley, L. C. McInnes, B. F. Smith, and H. Zhang. PETSc Web page, 2001. <http://www.mcs.anl.gov/petsc>.
- [8] G. Karypis and V. Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing* - <http://www.cs.umn.edu/metis>, 1998.

